favourite search engine, and you will likely find one of the many online man-page repositories. This will be less useful for Windows users though.

If you're already reaching for your browser though, there is no end to good learning resources for C. Simply a search will reveal great starting resources, so rather than reiterate those here we will share a three resources that teach something interesting.

**Create your own Kernel Module:**
If you ever wanted to get into really low-level programming, building your own driver for the Linux kernel, here is a guide that will help you. Instead of jumping into the deep end, this tutorial showcases a very simple kernel module that simply creates a /dev/reverse device in Linux that will reverse the word order of strings sent to it. Where you take it from there is up to you.

*http://www.linuxvoice.com/be-a-kernel-hacker/*

**Learn about Memory Allocators, possibly write your own:**
Believe it or not, someone actually has to code the bits of software that actually allocate, deallocate and keep track of the memory used by variables you use in your programs. Since creating and destroying variables happens a LOT in any software, speeding up this process can have a large impact on software performance, especially if the way you use variables is special, for instance if you are developing a game. These articles give an overview of this bit of software.

*http://jamesgolick.com/2013/5/15/memory-allocators-101.html*
*http://jamesgolick.com/2013/5/19/how-tcmalloc-works.html*

**Add a new feature to Python by modifying its code:**
If you've ever wondered how the features of a programming language map to actual code in its compiler or interpreter, this article is what you are looking for. It's written by a core Python contributor, and takes you through adding a new statement to the Python syntax by modifying its C code.

*http://eli.thegreenplace.net/2010/06/30/python-internals-adding-a-new-statement-to-python/*